

TASK 1 :

1. UCB :

- Code for UCB :

Python

```
class UCB(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # You can add any other variables you need here
        # START EDITING HERE
        self.counts = np.zeros(num_arms)
        self.values = np.zeros(num_arms)
        self.total_pulls = 0
        # END EDITING HERE

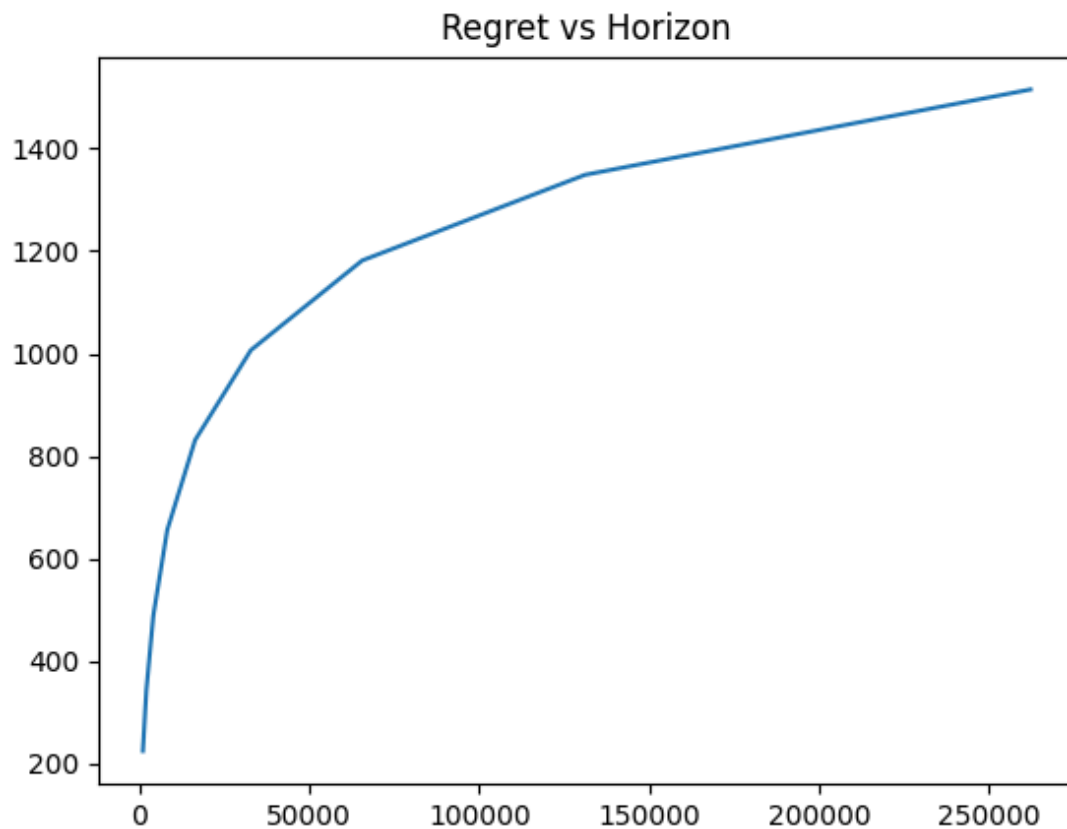
    def give_pull(self):
        # START EDITING HERE
        self.total_pulls+=1
        for a in range(self.num_arms):
            if(self.counts[a] == 0):
                return a
        ucb_values = self.values +
np.sqrt((2*math.log(self.total_pulls))/self.counts)
        return np.argmax(ucb_values)
        # END EDITING HERE

    def get_reward(self, arm_index, reward):
        # START EDITING HERE
        self.counts[arm_index]+=1
        n = self.counts[arm_index]
        value = self.values[arm_index]
        self.values[arm_index] = ((n-1)*value+reward)/n
        # END EDITING HERE
```

- Explanation :
 - Initialization
 - give_pull() :
 - Increment the total pulls counter
 - If any arm has never been pulled, pull it immediately
 - This ensures each arm is tried at least once for initial exploration
 - Compute Upper Confidence Bound (UCB) for each arm
 - Pull the arm with the highest UCB value
 - get_reward() :

- Increment the number of times this arm has been pulled
- Get the updated count for this arm
- Get the previous mean reward of this arm
- Update the mean reward incrementally

- Graph for UCB



2. KL_UCB :

- Code for KL_UCB :

Python

```
class KL_UCB(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # You can add any other variables you need here
        # START EDITING HERE
        self.counts = np.zeros(num_arms)
        self.values = np.zeros(num_arms)
```

```

self.e = 1e-6
self.c = 3

def kl_divergence(self,p,q):
    eps = 1e-12
    p = np.clip(p,eps,1-eps)
    q = np.clip(q,eps,1-eps)
    return (p*math.log(p/q)+((1-p)*math.log((1-p)/(1-q))))

    # END EDITING HERE

def give_pull(self):
    # START EDITING HERE
    for a in range(self.num_arms):
        if self.counts[a] == 0:
            return a

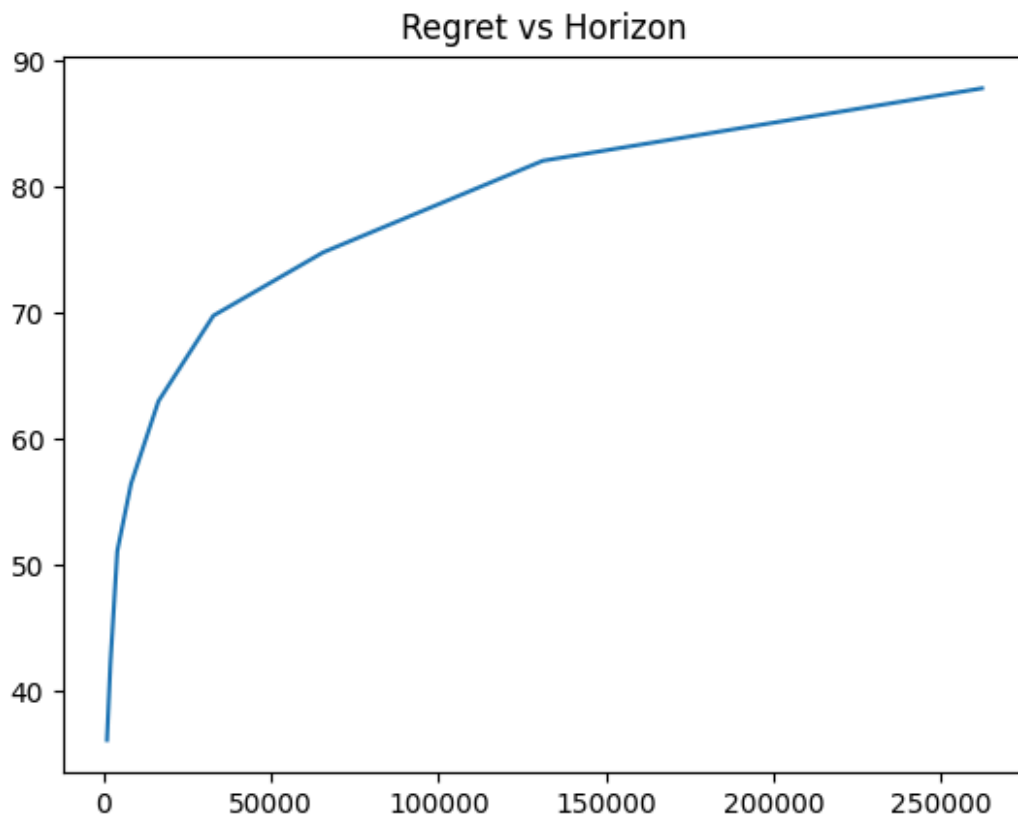
    t = int(np.sum(self.counts))
    ucb_values = np.zeros(self.num_arms)
    for a in range(self.num_arms):
        p = self.values[a]
        low, high = p, 1.0
        rhs =
(math.log(t)+self.c*math.log(math.log(max(t,2))))/self.counts[a]
        for _ in range(25):
            q = (low+high)/2
            kl = self.kl_divergence(p,q)
            if kl > rhs:
                high = q
            else:
                low = q
        ucb_values[a] = low
    return np.argmax(ucb_values)
    # END EDITING HERE

def get_reward(self, arm_index, reward):
    # START EDITING HERE
    self.counts[arm_index]+=1
    n = self.counts[arm_index]
    value = self.values[arm_index]
    self.values[arm_index] = ((n-1)*value+reward)/n
    # END EDITING HERE

```

- Explanation :

- Initialization
 - `kl_divergence()` :
 - KL Divergence measures the difference between two probability distributions
 - `np.clip` ensures probabilities stay in $[\text{eps}, 1-\text{eps}]$ to avoid $\log(0)$
 - Used in KL-UCB to compute the confidence bound based on information theory, rather than a simple square root like in standard UCB
 - `give_pull()` :
 - Pull each arm once to initialize (just like standard UCB)
 - Use binary search to find the largest q
 - This q becomes the KL-UCB value for that arm
 - Binary search ensures precision and avoids expensive numerical optimization
 - Pull the arm with the highest KL-UCB value
 - `get_reward()` :
 - Works exactly like in standard UCB, update the running mean reward after receiving a reward
- Graph for KL_UCB



3. Thompson Sampling :

- Code for Thompson Sampling :

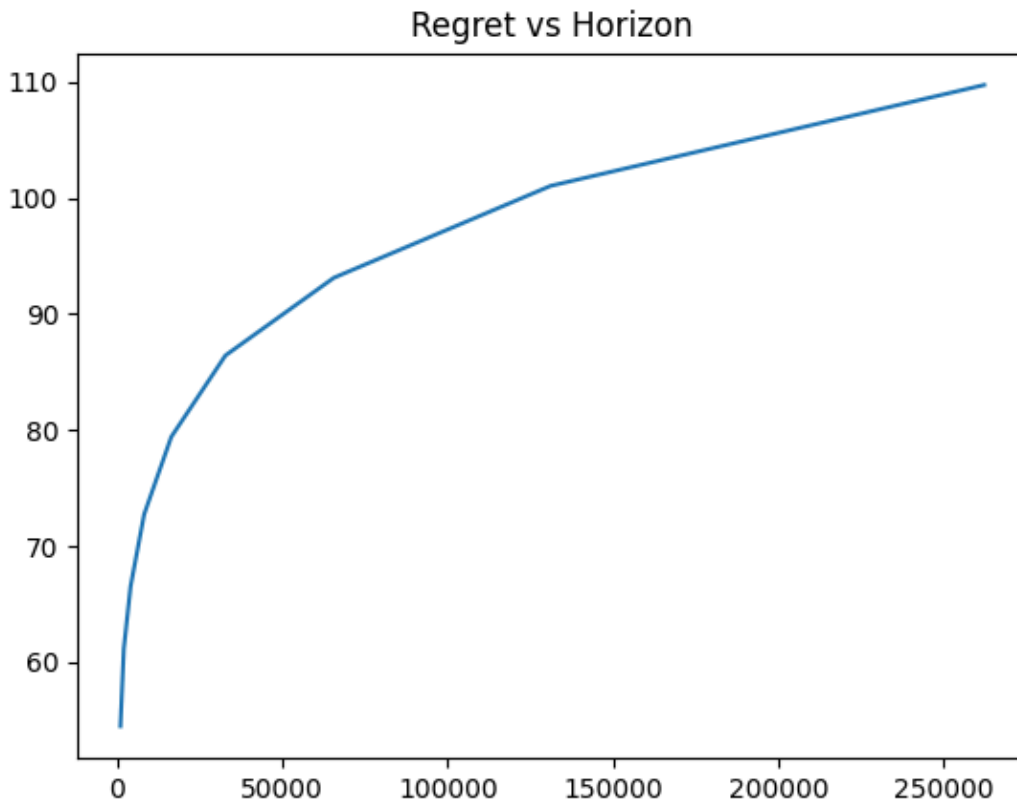
Python

```
class Thompson_Sampling(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # You can add any other variables you need here
        # START EDITING HERE
        self.st_a = np.zeros(num_arms)
        self.ft_a = np.zeros(num_arms)
        # END EDITING HERE

    def give_pull(self):
        # START EDITING HERE
        # Sample from Beta distribution for each arm
        samples = np.random.beta(self.st_a+1, self.ft_a+1)
        mask = samples == samples.max()
        return np.random.choice(np.arange(len(samples))[mask])
        # END EDITING HERE

    def get_reward(self, arm_index, reward):
        # START EDITING HERE
        if reward == 1:
            self.st_a[arm_index] += 1
        else:
            self.ft_a[arm_index] += 1
        # END EDITING HERE
```

- Explanation :
 - Initialization
 - give_pull() :
 - Sample a value from the posterior Beta distribution for each arm
 - samples → array of sampled probabilities, one for each arm
 - Identify arm(s) with the maximum sampled value (mask)
 - If multiple arms have the same maximum, randomly choose among them using np.random.choice
 - get_reward() :
 - After pulling an arm, update the posterior distribution for that arm :
 - reward = 1 → increment successes (st_a)
 - reward = 0 → increment failures (ft_a)
- Graph for Thompson Sampling :



TASK 2 :

The Policy :

The policy keeps a running estimate of how much damage each door is likely to take from a hit, using past observations to make these estimates more reliable. It starts by exploring a small set of doors that seem most promising, making sure each of them is tried at least once. After this initial exploration, the policy either:

- chooses a door that has a good chance of breaking in a single hit, or
- picks the door that is expected to take the fewest hits to break based on current estimates.

By using these estimates and focusing on the most promising doors, the policy avoids wasting strikes on doors that are unlikely to break quickly and prioritizes actions that are most likely to finish the task efficiently

Reasoning Behind the Approach :

- Since the damage is Poisson, the sum of several hits is a predictable pattern, which means we can reason about the total damage a door will take after multiple strikes

- If the door has a health H and the average damage per hit is “ u ” we can estimate the number of hits needed to break is roughly H/u
- Using gamma priors with Poisson damage makes updating our estimates straightforward. Each time we hit a door we can update our belief about its average damage.

Algorithm Overview :

- Each door is initialized :

Python

```
self.eps = 1e-12
self.health = np.full(K, 100, dtype=float) # health of each door is
hardcoded
self.a0 = 1.0
self.b0 = 1.0
self.exploit = K
self.fin_thresh = 0.12
self.xplr_setsize = max(1, int(np.ceil(K*1/3)))
self.xplr_set = None
self.exploitation = False
```

- Posterior parameters and posterior mean is computed to be used as damage estimates

Python

```
def posterior_params(self, a):
    alpha = self.a0+self.sums[a]
    beta = self.b0+max(1, int(self.counts[a]))
    return alpha, beta

def posterior_mean(self, alpha, beta):
    return alpha/beta
```

- Poisson CDF is used to calculate the finishing probability

Python

```
def poisson_cdf(self, lam, k):
    if k<0:
        return 0.0
    term = np.exp(-lam)
    s = term
```

```

for i in range(1,k+1):
    term*=lam/i
    s+=term
return min(1.0,s)

```

- Exploration Set : Set of promising arms to explore first
 - Compute posterior mean of each arm
 - Compute expected hits to break
 - If no arm is pulled yet or expected steps for each is same, pick a random subset
 - Else pick an arm with the smallest expected steps
 - Set exploitation to False, the arm is still in exploration mode

Python

```

def ensure_xplr_set(self):
    if self.xplr_set is not None:
        return
    post_means = np.array(list(map(lambda a:
self.posterior_mean(*self.posterior_params(a)), range(self.K))))
    expected_steps = self.health / np.maximum(post_means, self.eps)
    if np.all(self.counts == 0) or np.isclose(np.std(expected_steps), 0.0):
        self.xplr_set = self.rng.choice(self.K, size=self.xplr_setsize,
replace=False)
    else:
        self.xplr_set = np.argsort(expected_steps)[:self.xplr_setsize]
        self.exploitation = False

```

- Arm selection
 - Seed explore

Python

```

self.ensure_xplr_set()
if not self.exploitation:
    unpulled_arm = [a for a in self.xplr_set if self.counts[a] == 0]
    if unpulled_arm:
        post_means_local = np.array(list(map(lambda a:
self.posterior_mean(*self.posterior_params(a)), unpulled_arm)))
        expected_steps = np.array(list(map(lambda a_pm:
self.health[a_pm[0]] / max(a_pm[1], self.eps), zip(unpulled_arm,
post_means_local))))

```



```

        return int(unpulled_arm[int(np.argmin(expected_steps))])
    else:
        self.exploitation = True
        post_means = np.array(list(map(lambda a:
self.posterior_mean(*self.posterior_params(a)), range(self.K))))

```

- Compute finish probabilities

Python

```

finish_probs = []
for a in range(self.K):
    threshold = int(np.ceil(self.health[a]))-1
    prob = 1.0 - self.poisson_cdf(post_means[a], threshold)
    finish_probs.append(prob)

finish_probs = np.array(finish_probs)
finish_thresh = max(0.01, self.fin_thresh*self.exploit/max(1, t))
best_finish = int(np.argmax(finish_probs))
if finish_probs[best_finish] >= finish_thresh:
    return best_finish

```

- Fallback to expected steps greedy

Python

```

pm_indices = self.health/np.maximum(post_means, self.eps)
return int(np.argmin(pm_indices))

```

- Updating after pulling the arm
 - Updates self.counts[arm] and self.sums[arm] used by posterior
 - Reduces remaining health of the doors
 - If in initial exploration phase, check whether every arm in the explore set is pulled at least once, if yes then switch to exploitation i.e self.exploitation = False

Python

```

def update(self, arm: int, reward: float):
    super().update(arm, reward)
    self.health[arm] -= reward

```

```

if self.xplr_set is not None and not self.exploitation:
    all_pulled = True
    for a in self.xplr_set:
        if self.counts[a] <= 0:
            all_pulled = False
            break
    if all_pulled:
        self.exploitation = True

```

The above Policy :

- Uses Bayesian approach to get reliable estimates of each door's average damage, so early random hits don't mislead the strategy
- Instead of wasting time on every door, it focuses on a small set of promising doors to quickly figure out which ones are worth striking
- If a door looks like it could break in a single hit, the policy goes for it immediately, taking advantage of high-probability chances to finish fast
- Otherwise, it targets the door that is expected to require the fewest strikes to break, minimizing the total steps needed

Practical Effectiveness :

- Concentrating on a subset of promising doors reduces computational and sample inefficiency
- Choosing doors based on how likely they are to finish quickly is a simple strategy, but it works really well in practice when the damage follows a Poisson distribution

Autograder Output :

```

• (env747) PS D:\IITB\cs747\assignment_1> python .\autograder.py --task 2
===== Task 2 =====
=== Autograder: Task 2 ===
task2-1.txt: steps=90.6 threshold=93 -> PASS
task2-2.txt: steps=137.1 threshold=150 -> PASS
task2-3.txt: steps=100.4 threshold=105 -> PASS
task2-4.txt: steps=66.4 threshold=73 -> PASS
task2-5.txt: steps=118.2 threshold=125 -> PASS
task2-6.txt: steps=85.9 threshold=98 -> PASS
task2-7.txt: steps=73.6 threshold=87 -> PASS
task2-8.txt: steps=124.2 threshold=150 -> PASS

Summary: 8/8 passed

Task 2 Score: 1.50/1.50 marks (8/8 public testcases passed)
=====
Time elapsed: 40.09 seconds
• (env747) PS D:\IITB\cs747\assignment_1> 

```

TASK 3 :

KL_UCB_Optimized V/S KL_UCB_Standard

Optimization Techniques :

- Initial forced exploration

Python

```
def give_pull(self):
    if np.min(self.counts) == 0:
        for a in range(self.num_arms):
            if self.counts[a] == 0:
                self.current_arm = a
                self.pull_count = 1
                self.current_batch = 1
                return a
```

- Batched updates : Instead of recomputing all arms unlike in standard kl_ucb the new algorithm :

Python

```
if self.pull_count < self.current_batch:
    self.pull_count+=1
return self.current_arm
```

Python

```
self.ucb_values[self.current_arm] =
self.compute_ucb(self.current_arm,t)
self.current_arm = int(np.argmax(self.ucb_values))
```

- caches ucb_values for all arms
- recomputes the ucb only for the currently tracked arm when a batch of repeated pull ends
- continues pulling the same arm for current batch thus the cost of single ucb calculation is reduced significantly
- Adaptive Batching :

Python

```
n = int(self.counts[self.current_arm])
if n <= 0:
    self.current_batch = 1
else:
    exponent = int(math.floor(math.log2(n+1)))
    self.current_batch = min(self.max_batch, max(1, 2**exponent))
self.pull_count = 1
```

- current_batch grows with the arm's count, so arms with many observations are pulled in larger batches.
- larger batches are safe once an arm estimate is stable
 - early phase : small batches, ensures responsiveness during exploration
 - later phase : large batches, amortize compute after arm is stable
- compute_ucb reduces the number of binary iterations dynamically, which lowers the number of KL evaluations per ucb computation
- Reduced Binary iterations

Python

```
base_iter = int(max(3, math.log(self.horizon)))
num_iterations =
min(10, base_iter - int(math.log(n+1)/2) - int(math.log(self.num_arms)/2))
for _ in range(num_iterations):
    q = (low+high)/2
    if self.kl_divergence(p, q) > rhs:
        high = q
    else:
        low = q
```

Regret Dynamics :

- In the earlier stage, small batches, the algorithm still updates ucbs frequently → almost no impact on regret
- Later stage, batch becomes larger, we might pull an arm a few extra times before noticing another arm has a slightly higher UCB.
- This can introduce some extra regret, but :
 - For well separated arms this effect is negligible
 - For very close arms the extra pulls add a small multiplicative factor in the regret

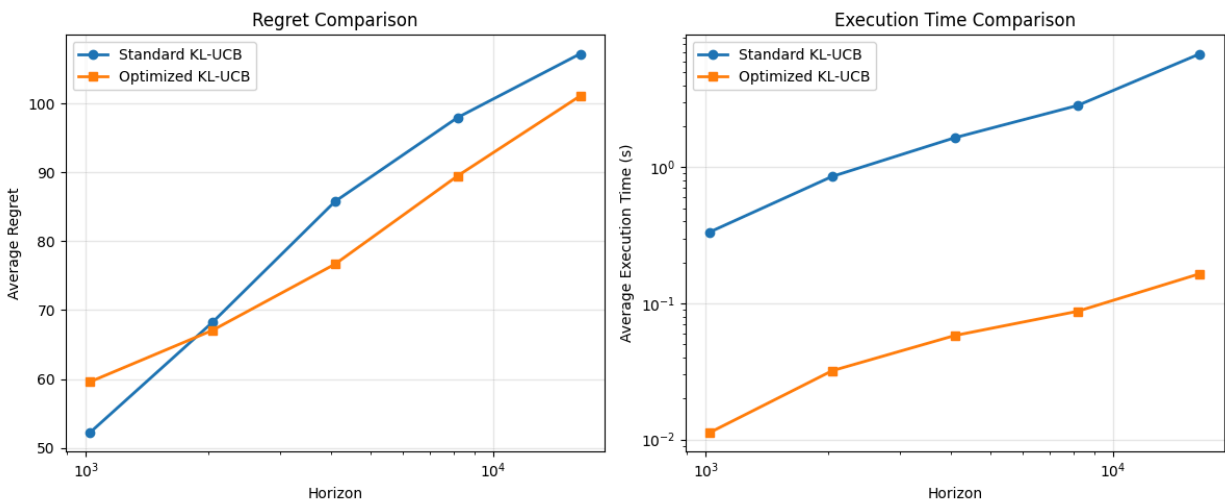
Graph Analysis :

Regret :

- At small horizons Standard KL-UCB has slightly higher regret than Optimized KL-UCB, this suggests that the batching and approximate UCB in Optimized KL-UCB may actually help in early exploration by reducing variance in arm selection
- As horizons increases the optimized KL_UCB regret will catch up with the standard KL_UCB and will surpass it as we can see in the upward going trend
- The slope of the regret is same which means that the regret for both are asymptotically same

Performance :

- Optimized KL_UCB is dramatically faster than standard
- On a logarithmic y-axis scale, execution time for optimized remains extremely low (~0.005–0.1 s), while standard KL-UCB grows roughly linearly with horizon
- This huge speedup comes from the batched pulls and lazy UCB updates
- As the horizon grows, standard KL-UCB must compute UCBs for all arms at each time step, making runtime grow almost linearly with T



KL_UCB_Bonus V/S KL_UCB_Standard

Optimization Techniques :

- Vectorization :

Python

```
def compute_ucb(self, t):
    pulled_ai = np.where(self.counts>0)[0]
    ucb_values = np.full(self.num_arms, float('inf'))
    if len(pulled_ai) == 0:
        return ucb_values
    p = self.values[pulled_ai]
    n = self.counts[pulled_ai]
```

```

rhs = (math.log(t)+self.c*math.log(math.log(max(t,2))))/n
low = p.copy()
high = np.ones_like(p)
for _ in range(25):
    q = (low+high)/2
    kl_div = self.kl_divergence(p,q)
    mask = kl_div>rhs
    high[mask] = q[mask]
    low[~mask] = q[~mask]
ucb_values[pulled_ai] = low
return ucb_values
#END EDITING HERE

```

- Computing UCB values for all active arms in vectorized form, using NumPy arrays instead of Python loops
- Selecting only arms that have been pulled at least once
- Binary Search in Bulk :

Python

```

low = p.copy()
high = np.ones_like(p)
for _ in range(25):
    q = (low+high)/2
    kl_div = self.kl_divergence(p,q)
    mask = kl_div>rhs
    high[mask] = q[mask]
    low[~mask] = q[~mask]

```

- The KL_UCB_Bonus performs binary search simultaneously on all active arms using vector operations
- This reduces Python-level looping overhead and allows NumPy's fast C-level computations to handle the updates efficiently
- Rewards and counts are updated incrementally, thus calculating from scratch is avoided

Python

```

def get_reward(self, arm_index, reward):
    #START EDITING HERE
    self.counts[arm_index] += 1

```

```
n = self.counts[arm_index]
self.values[arm_index]+=(reward-self.values[arm_index])/n
#END EDITING HERE
```

Regret Dynamics :

- Both algorithms pick the same arms in the same order for the same random seed
- Minor differences in numerical precision might slightly alter floating-point UCB values, but the cumulative regret trajectory remains nearly identical

Performance Dynamics :

- Optimized KL-UCB achieves 20–30× speedup, especially noticeable at large horizons
- Most speedup comes from vectorized UCB computation and skipping unnecessary calculations

Graph Analysis :

Standard KL_UCB Trajectories :

- Regret grows roughly logarithmically with time (typical for KL-UCB)
- There's slight variation between simulations because each simulation experiences different stochastic rewards
- Early steps have sharp increases in regret (initial exploration phase), then curves flatten as the algorithm identifies the best arm

Bonus KL_UCB Trajectories :

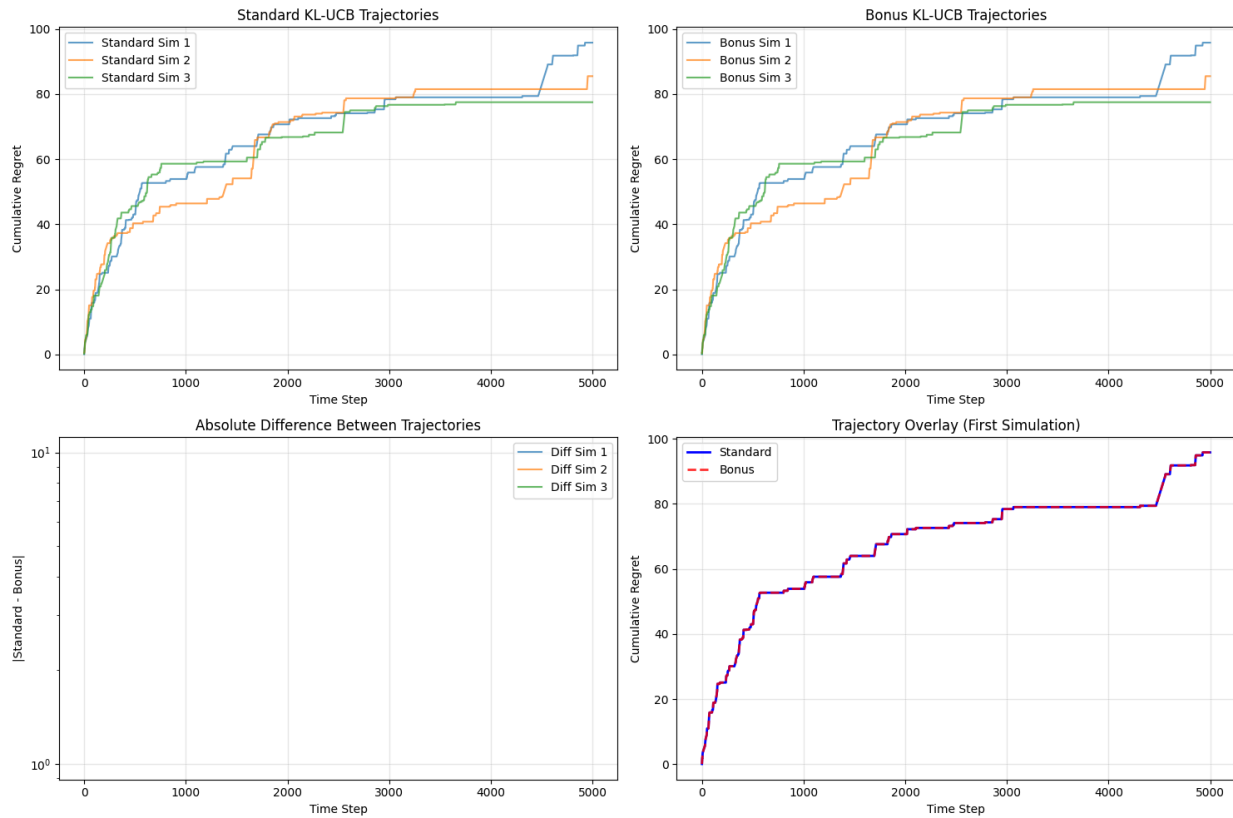
- Trajectories are nearly identical to the standard KL-UCB simulations
- Differences are negligible, mainly due to floating-point precision during vectorized UCB calculations
- Regret pattern (exploration then exploitation) matches exactly with Standard KL-UCB

Absolute Difference Between Trajectories :

- Differences are none/ very small
- The Bonus KL-UCB produces effectively identical cumulative regret compared to Standard KL-UCB

Trajectory Overall :

- Overlays Standard (blue line) and Bonus (red dashed line) cumulative regret for the first simulation
- Lines are completely overlapping, confirming that for a fixed seed, the Bonus variant replicates the exact same action-reward sequence



References :

- https://ala2019.vub.ac.be/papers/ALA2019_paper_9.pdf
- https://bookdown.org/kevin_davisross/bayesian-reasoning-and-methods/poisson.html
- <https://www.georgeho.org/bayesian-bandits/>
- <https://hamsabastani.github.io/greedybandit.pdf>
- <https://allendowney.github.io/ThinkBayes2/chap18.html>
- <https://arxiv.org/pdf/2310.14129>
- [https://www.researchgate.net/publication/379368127 Comparative analysis of the KL-UCB and UCB algorithms Delving into complexity and performance](https://www.researchgate.net/publication/379368127_Comparative_analysis_of_the_KL-UCB_and_UCB_algorithms_Delving_into_complexity_and_performance)
- <https://arxiv.org/abs/1102.2490>